Multivariate Time Series Anomaly Detection Based on Multiple Spatiotemporal Graph Convolution

Shiming He, Qingqing Guo, Genxin Li, Kun Xie, *Member, IEEE*, and Pradip Kumar Sharma, *Senior Member, IEEE*

***Abstract*— Multivariate time series anomaly detection (MTSAD) plays a crucial role in the Internet of Things (IoT), identifying device malfunction or system attacks. Graph neural networks (GNNs) are widely applied in MTSAD to capture the spatial features among sensors. However, GNN requires an explicit graph structure and cannot work when spatial relationships are lacking or sensor dependencies are unknown. Hence, graph structure learning (GSL) emerges as a promising solution, which learns graph structures with the downstream tasks without requiring spatial relationships. However, a general cascade effect occurs in the industrial scene. Moreover, an attack event changes sensor data sequentially instead of simultaneously due to the multiple serial process that occurs (i.e., delay correlation). Existing GSL-based anomaly detection methods fail to tackle delay correlation and focus on low prediction errors. However, the low prediction error distribution is always dispersed, making it difficult to differentiate between normal and abnormal samples based on a threshold. Therefore, we propose an MTSAD based on multiple spatiotemporal graph convolution (MSTGAD). MSTGAD uses dynamic time warping (DTW) distance as prior knowledge instead of Euclidean, guiding graph learning to capture delay correlations effectively. MSTGAD designs an ensemble predictor, which uses two subpredictors with a shared and learnable graph to ensure high prediction accuracy while maintaining stable anomaly scores. Furthermore, we conduct extensive experiments to evaluate the MSTGAD method's effectiveness. Our method outperforms existing GSL-based methods in anomaly detection on four publicly available real-world datasets, demonstrating our proposed approach's effectiveness. Compared with the best GSL-based method, MSTGAD achieves an additional 0.83% improvement on average.**

***Index Terms*— Anomaly detection, graph neural networks (GNNs), graph structure learning (GSL), Internet of Things (IoT), multivariate time series (MTS).**

NOMENCLATURE

X	Multivariate time series.
x^i	ith indicator values.
x_t	Indicator values at timestamp t.
X_t	Historical window of length ω at timestamp t.
N	Number of indicators.
M	Number of timestamps.
$\widehat{y}_t$	Label of timestamp t.
G	Graph.
A	Adjacency matrix.
$\mathrm{DTW}(x^i, x^j)$	DTW distance between indicators i and j.
A^{DTW}	Adjacency matrix of the DTW prior graph.
ε	Threshold of the DTW distance in prior graph.
g	Samples from the Gumbel distribution.
π_i	Probabilities of ith class.
τ	Temperature parameter.
A^{FPM}	Adjacency matrix of the fully parameterized graph.
$\Gamma *_{\mathcal{T}} X$	Temporal convolutional network.
$\Theta *_{\mathcal{G}} X$	Graph convolutional network.
$\odot$	Hadamard product of elements.
Θ^l	Spectral kernel of the GCN.
$\widehat{x}_t^{\mathrm{STGCN}}$	Prediction from the STGCN subpredictor at timestamp t.
$H_{(t)}$	Hidden features at timestamp t.
$\widehat{x}_t^{\mathrm{DCRNN}}$	Prediction from the DCRNN subpredictor at timestamp t.
$\widehat{x}_t$	Prediction from the ensemble predictor at timestamp t.
λ_1	Prediction weight parameter.
λ_2	Regularization amplitude parameter of graph learning.
$Err_i(t)$	Prediction error at timestamp t for sensor i.
$s(t)$	Anomaly score at timestamp t.

I. INTRODUCTION

IN RECENT years, more critical infrastructures have been deployed by various Internet-of-Things (IoT) sensors.

Received 30 May 2024; revised 20 August 2024; accepted 6 September 2024. Date of publication 22 November 2024; date of current version 26 November 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62272062 and Grant 62025201; in part by the Science and Technology Innovation Program of Hunan Province under Grant 2023RC3139; in part by the Scientific Research Fund of Hunan Provincial Transportation Department under Grant 202143; in part by the Open Fund of Key Laboratory of Safety Control of Bridge Engineering, Ministry of Education, Changsha University of Science and Technology, under Grant 21KB07; and in part by the Open Research Fund of the Hunan Provincial Key Laboratory of Network Investigational Technology under Grant 2018WLZC003. The Associate Editor coordinating the review process was Dr. Ke Feng. *(Corresponding author: Kun Xie.)*

Shiming He, Qingqing Guo, and Genxin Li are with the School of Computer and Communication Engineering and the Hunan Provincial Key Laboratory of Intelligent Processing of Big Data on Transportation, Changsha University of Science and Technology, Changsha 410114, China (e-mail: smhe_cs@csust.edu.cn; guoqingqing@stu.csust.edu.cn).

Kun Xie is with the College of Computer Science and Electronics Engineering and the Ministry of Education Key Laboratory of Fusion Computing of Supercomputing and Artificial Intelligence, Hunan University, Changsha 410082, China (e-mail: xiekun@hnu.edu.cn).

Pradip Kumar Sharma is with the Department of Computing Science, University of Aberdeen, AB24 3UE Aberdeen, U.K. (e-mail: Pradip.sharma@abdn.ac.uk).

Digital Object Identifier 10.1109/TIM.2024.3493890

These sensors, such as water treatment, lithium-ion batteries [1], and nanogenerators [2], continue to generate substantial amounts of time series data that record system statuses. The data generated by these sensors are known as multivariate time series (MTS) data. Developing effective anomaly detection technologies to automate the identification of abnormal system behavior and promptly trigger alerts is a necessary and urgent task. This is because abnormal MTS data changes indicate a system attack or equipment failure. If the anomaly is not discovered and dealt with immediately, the whole system may fail, leading to economic losses [3]. Therefore, the MTS anomaly detection (MTSAD) task plays an important role in the IoT [4].

In the IoT, sensors interact with and depend on each other. In recent years, graph neural networks (GNNs) [5] have been incorporated into MTSAD to capture the relationships between sensors. GNNs require an explicit graph structure. Therefore, the existing GNN-based anomaly detection methods use a fixed graph structure derived from the prior domain expert knowledge. However, sensor graph structures or dependency information is expensive and not easily accessible in most real-world scenarios.

Graph structure learning (GSL) [6] is a promising solution when expert knowledge is lacking, sensor dependencies are unknown, and access costs are high [7]. It learns the optimal graph structure of the dependencies among sensors to match downstream anomaly detection tasks. Graph deviation network (GDN) [8] first introduced GSL into anomaly detection, resulting in two mainstream GSL methods being used in anomaly detection currently: node feature-based similarity [8], [9], [10], [11], [12], [13], [14], [15] and fully parameterized GSL [16], [17], [18], [19], [20]. The former generates a graph using the learnable node embedding similarities, while the latter regards the adjacency matrix's elements as free and learnable parameters. In addition, fully parameterized GSL offers more flexibility than the node feature-based similarity method. Although GSL can learn the relationships among sensors [15], the existing GSL-based anomaly detection method still encounters several challenges.

1) *The Existing Prior Graph Similarity Methods Cannot Tackle With the Delay Correlation Within Time Series Data:* The existing GSL-based anomaly detection methods usually use prior graphs [21] to guide GSL. The prior graph in TS-GAT [11] and GSLAD [20] is generally generated using sensor data similarities, and the main similarity methods include the cosine and Euclidean distance approaches, which only compare the same timestamp values between sensors. However, a general cascade effect in the industrial field exists, mainly due to the presence of certain distances in space between industrial sensors. Moreover, an attack even changes sensor data sequentially instead of simultaneously. The cosine or Euclidean distance approaches ignore the similarities between different timestamp values noise into the prior graph.
2) *Existing Anomaly Detection Methods Focus on Low Prediction Error. However, Models With High Prediction Accuracy Have Large Prediction Error Variances, Resulting in a Dispersed Distribution. This Makes Distinguishing Between Normal and Abnormal Samples Based on a Threshold Challenging, Thereby Diminishing Anomaly Detection Performance:* Most existing GSL-based anomaly detection methods focus on the prediction error generated by GNNs and treat timestamps with a large prediction error as anomalies. They generally select prediction models with a low prediction error and high overall accuracy [19], [20]. However, the prediction and anomaly detection tasks differ. Although the overall prediction accuracy is high, the anomaly detection effectiveness may not be satisfactory. In addition, models with high overall prediction accuracy may exhibit inconsistent prediction errors (i.e., anomaly scores) with large variances. This means that the prediction error at certain timestamps is low, while being significantly high at others. Consequently, distinguishing between normal and abnormal samples based on the prediction error values becomes challenging, leading to unsatisfactory anomaly detection performance.

To address the aforementioned problems, we propose MTSAD based on multiple spatiotemporal graph convolution (MSTGAD). We leverage the dynamic time-warping (DTW) distance to establish a DTW graph as prior knowledge, guiding the learning process to capture delay correlations effectively. In addition, we adopt a fully parameterized approach to construct the learnable graph, substantially reducing time overhead. Moreover, we combine the diffusion convolutional recurrent neural network (DCRNN) and spatiotemporal graph convolutional network (STGCN) with a shared and learnable graph for ensemble prediction. This strategy ensures precise predictions and stabilizes the anomaly scores. This article's main contributions can be summarized as follows.

1) To effectively address delay correlation, we generate a prior graph from raw data using DTW distance instead of Euclidean to accurately guide GSL.
2) To concentrate and stabilize the anomaly scores necessary for anomaly detection, we integrate multiple spatiotemporal predictors with a shared and learnable graph instead of focusing solely on the high overall prediction accuracy. This strikes a balance between prediction accuracy and anomaly score stability.
3) We conduct extensive experiments to evaluate the proposed method's efficiency and effectiveness. On four real-world and publicly available datasets, our method achieves state-of-the-art anomaly detection results. Compared with the best GSL-based method, MSTGAD achieves an additional 0.83% improvement on average.

This article's remaining sections are organized as follows. Section II presents an overview of related works. Section III provides the preliminary knowledge, problem definition, and research motivation. Section IV comprehensively explains our approach, including a general overview and an individual components discussion. In Section V, we analyze the model's performance and efficiency. Finally, we conclude by summarizing our findings and discussing future work in Section VI.

II. Related Work

The existing works on MTSAD can be classified into three categories. The first type usually uses temporal features to detect anomalies, while the second focuses on MTS spatial features. The third type detects anomalies with unknown spatial relationships.

A. Temporal Feature-Based Anomaly Detection

Long short-term memory (LSTM) is widely used in MTSAD due to its ability to process sequence data. For example, LSTM-NDT [22] uses LSTM to achieve a high prediction performance while maintaining the whole system's interpretability. It uses a nonparametric, dynamic, and unsupervised threshold method to detect anomalies. In contrast, MSCRED [23] constructs a multiscale feature matrix and uses conv-LSTM neural networks to encode the correlation between sensors and reconstruct multiscale feature matrix. The residual feature matrix facilitates anomaly detection and diagnosis. MAD-GAN [24] uses LSTM as the basic model in the generative adversarial network framework, capturing the time series temporal correlation, extracting potential interactions, and detecting anomalies using discrimination and reconstruction. LSTM-DAE [25] uses a model trained on operational cycle signals to select outlier candidate points in the first stage. Then, it uses another model trained on sensor signals exploiting temporal continuity to detect abnormal events among the candidate points in the second stage. In comparison, LSTM-VAE [26] introduces multimodal observation and time dependence into the potential space, reconstructs the expected distribution, and evaluates the anomalies based on the reconstructed anomaly scores.

Other recurrent neural networks (RNNs) beside LSTM are also applied in MTSAD. For example, OmniAnomaly [27] uses a stochastic RNN to capture the robust representations of normal patterns and reconstruct observations. DAGMM [28] uses a depth self-coding and Gaussian mixture model to produce low-dimensional representations and reconstruction errors to detect anomalies.

Although these temporal feature-based methods achieve adequate anomaly detection results, they ignore time series correlations.

B. Spatial Feature-Based Anomaly Detection

Graph attention networks (GATs) extract graph data features and use them to capture the correlation between indicators and predict the time-dependent relationships. For example, MTAD-TF [29] captures the temporal and spatial patterns produced by multiscale convolution networks and GAT. MTAD-GAT [30] obtains the correlation between different univariate time series and the temporal dependence within each time series produced by GAT. It integrates the prediction and reconstruction tasks to detect anomalies. In contrast, Arvalus [31] treats system components as nodes, and their dependencies and positions as edges, thereby improving the identification and location of anomalies. MDGAT [32] uses a novel architecture based on the GAT with multihead dynamic attention to learn the dependencies between sensors in temporal and spatial spaces.

Moreover, DVGCRN [33] captures uncertain interrelationships in MTS and detects anomalies based on the reconstruction results. GReLeN [34] uses VAE and GCN to learn relationships among MTS for reconstructing graphs and achieve anomaly detection.

GNNs rely on known graph structures. These methods assume that the relationships between indicators are known. However, the relationships between indicators are often unknown or the cost of obtaining them is high.

C. Unknown Spatial Relationship-Based Anomaly Detection

GSL has been used in the MTSAD field to capture spatial indicator features without graph structure. Two GSL approaches are typically used for anomaly detection: the node feature-based similarity and fully parameterized methods.

First, GDN [8] learns the relationship graph between sensors based on learnable node features and identifies data that deviate significantly from the expected distribution. Building on GDN, HGTMAD [9] leverages GCNs and the transformer to extract spatial and temporal features independently. In contrast, FuSAGNet [10] uses a fused sparse self-encoder graph to explicitly model the relationships between MTS data. It predicts and reconstructs future time series to detect anomalies. TS-GAT [11] uses prior knowledge from raw data to constrain graph learning. MEGA [12] and DPGLAD [13] learn a unidirectional graph for anomaly detection. GSC-MAD [14] uses aggregative graph updating, smooth, and sparse constraints to learn a stable graph structure. ACGSL [15] learns a graph for individual subsequences using the difference constraint between two consecutive subsequences, resulting in dynamic graphs.

GTA [16] introduces a fully parameterized GSL strategy and uses a transformer for predictions. It detects and diagnoses anomalies in the time series by evaluating the second norm of the difference between predicted and actual values. Subsequently, GLAD [17] combines GAT and a transformer to extract global and local features for anomaly detection. MGCLAD [18] uses two directed graphs to simultaneously learn inter- and intra-signal correlations. MEGLAD [19] exploits three GSL types to extract multiple correlation views. Finally, GSLAD [20] uses diffusion graph convolution networks to detect anomalies.

Node feature-based similarity GSL only learns a simple KNN graph, where the node degree is less than K. In contrast, fully parameterized GSL is more flexible. However, using a transformer for temporal prediction in GTA, GLAD, and MGCLAD increases the model's time complexity.

III. Problem Definition and Preliminaries

This article aims to explore anomaly detection in MTS. In this section, we elaborate on the basic concepts of anomaly detection in MTS and introduce our motivation.

A. Multivariate Time Series

MTS data comprises a large number of regularly spaced and uninterrupted samples, which are characterized by N indicators and M timestamps, denoted by $X = (x_1, x_2, \ldots, x_M)^T \in$

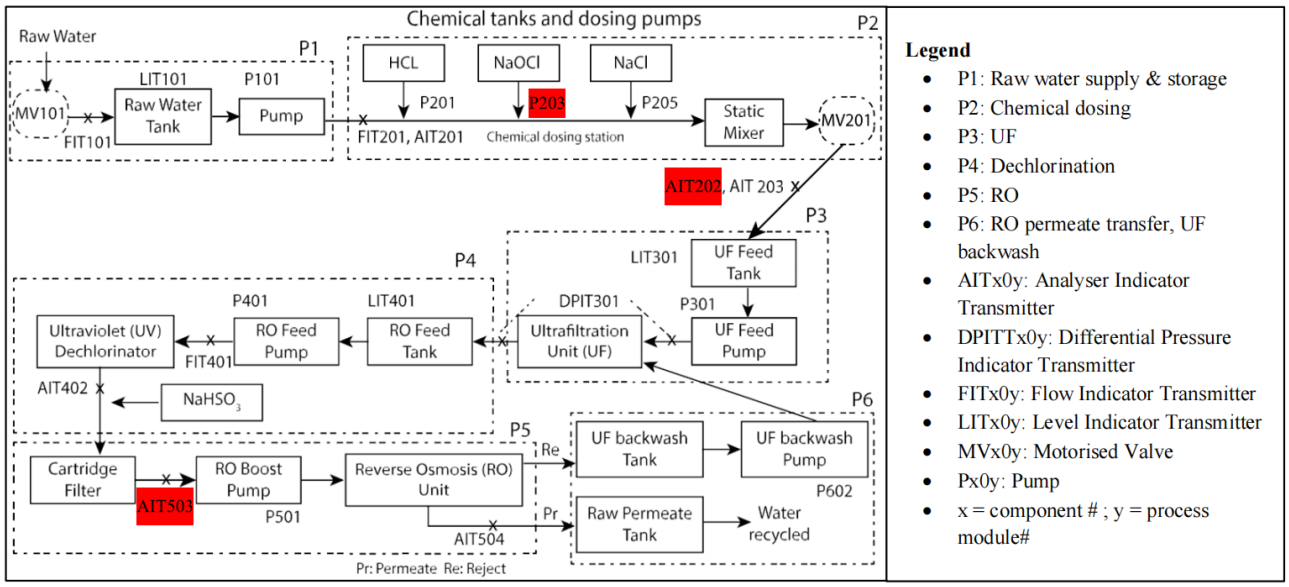

Fig. 1. Six-stage SWaT test bed processes are P1–P6. These processes include raw water supply and storage, chemical dosing, UF, dechlorination, RO, and RO permeate transfer, UF backwash. Each process contains several sensors named according to their function and process. For example, AIT-203 represents the third analyzer indicator transmitter in the P2 process. The three sensors marked in red (P-203, AIT-202, and AIT-503) are used as examples to explain delay correlation in Fig. 2.

$R^{M\times N}$. The ith indicator is defined as $x^i = (x_1^i, x_2^i, \ldots, x_M^i)$. In addition, the tth timestamp contains N indicator values, which is defined as $x_t = (x_t^1, x_t^2, \ldots, x_t^N)^T$. The history window on timestamp t with length ω is defined as a subsequence $X_t = (x_{t-\omega}, x_{t-\omega+1}, \ldots, x_{t-1})^T \in R^{\omega\times N}$. When the label y of timestamp t is 1, an anomaly has occurred at timestamp t. When the label y of timestamp t is 0, timestamp t is normal.

B. Anomaly Detection in MTS

MTSAD calculates the anomaly score of each timestamp t according to the historical subsequence X_t and records it as s_t. If the anomaly score s_t exceeds a predetermined threshold, an anomaly has occurred at timestamp t. This process is defined as follows:

$$\widehat{x}_t = f(X_t) \tag{1}$$

$$s_t = \varphi(x_t, \widehat{x}_t) \tag{2}$$

$$\widehat{y}_t = \begin{cases} 1, & \text{if } s_t > \text{Thr} \\ 0, & \text{if } s_t \leq \text{Thr} \end{cases} \tag{3}$$

where $f()$ is the prediction function, $\widehat{x}_t$ denotes the predicted value at timestamp t, x_t represents the ground truth, φ indicates the anomaly scores function, and Thr is the threshold.

C. Graph Structure Learning

Given the time series $\mathbf{X} \in R^{M\times N}$, GSL aims to obtain a graph $G = (V, E)$ and its topology or adjacency matrix $A \in R^{N\times N}$. In this instance, the graph nodes are the sensors that produce the indicators, and the edges E are the hidden sensor relationships. Furthermore, the adjacency matrix A stores the edge information in the graph, reflecting the underlying dependencies between indicators.

The adjacency matrix elements consist of 0 and 1. If $A_{i,j}$ is 1, an edge appears between nodes i and j. On the contrary, when $A_{i,j} = 0$, no edge occurs between nodes i and j.

D. Motivation

Generally, GSL-based MTSAD methods outperform current approaches. However, the existing methods encounter the following challenges.

1) Ignoring Delay Correlation in MTS: Typically, industrial systems are executed within multiple serial processes. For example, the Secure Water Treatment (SWaT) test bed includes six processes (P1–P6): raw water supply and storage, chemical dosing, ultrafiltration (UF), dechlorination, reverse osmosis (RO), and RO permeate transfer, UF backwash, as shown in Fig. 1. Each process contains several sensors that are named according to their functions and processes. For example, AIT-203 represents the third analyzer indicator transmitter in the P2 process. A process attack or fault can trigger a cascade effect on other processes, causing delays within the time series. As shown in Fig. 2, AIT-202 experienced an attack from timestamps 707 to 730, during which the P-203 sensor responded promptly. However, the AIT-503 sensor only starts responding at timestamp 760. This is because AIT-202 and P-203 are deployed during P2, while AIT-503 is deployed at P5. This is known as delay correlation. The existing prior graph generation methods generally obtain the time series similarity based on cosine or Euclidean distance, which calculates the sensor proximity at the same timestamp and ignores the similarity between different timestamps. This makes delay correlation challenging to tackle; therefore, these methods are not suitable for asynchronous time series data, leading to missing edges in the prior graph.

2) Existing Predictors Cannot Meet the Requirements of High Prediction Accuracy and Concentrated Anomaly Score

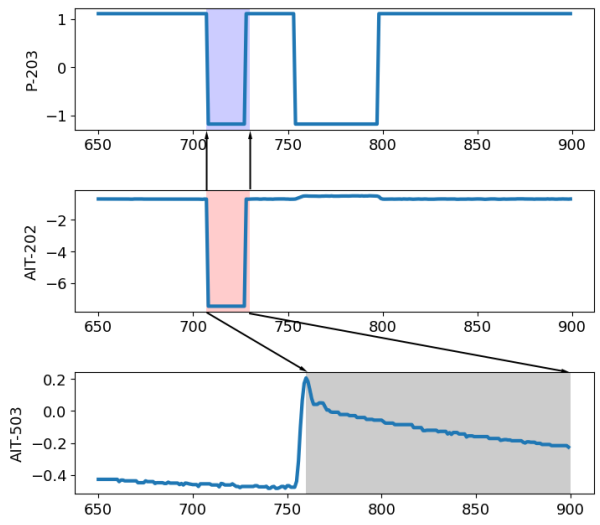

Fig. 2. Example of delay correlation in the SWaT dataset. We consider three sensors marked in red in Fig. 1: P-203 and AIT-202 in the P2 process, and AIT-503 at P5. During the timestamps marked in pink, the sensor AIT-202 experienced an attack, while the sensor P-203 responded during the same timestamps, marked in blue. On the other hand, the sensor AIT-503 responded during the subsequent timestamps marked in gray.

Distributions Simultaneously: DCRNN and STGCN are excellent spatiotemporal predictors.

DCRNN combines MTS with diffusion processes to capture spatial dependencies. Furthermore, it uses gated recurrent units to capture time dependencies. It also excels in prediction tasks by effectively capturing temporal and spatial dependencies, making it an ideal choice for achieving high accuracy and minimizing overall errors. Anomaly detection tasks aim to minimize prediction errors at normal timestamps and maximize those at abnormal timestamps. In addition, we expect the overall prediction error variance at normal timestamps to be small, and we strive for minimal anomaly score fluctuations. However, DCRNN exhibits wide anomaly score variances, ranging from 1 to 11, as shown in Fig. 3. According to the figure, the anomaly score exhibits a sudden increase around timestamp 8000. This can be attributed to the indicator (2B_AIT_002_PV) sharply rising from 0.05 to 267 during that timestamp. This variance significantly affects the anomaly detection task performance. The detailed sensor values in the Water Distribution (WADI) dataset can be found in Appendix.

In comparison, STGCN combines graph and causal convolution to extract temporal and spatial features. This network comprises convolutional structures that can be computed in parallel, resulting in faster speed. Therefore, STGCN is also an excellent spatiotemporal predictor. During the prediction task, STGCN has a low prediction accuracy. However, during the anomaly detection task, it exhibits relatively stable anomaly scores with low variance, ranging from 11 to 16, as shown in Fig. 4. The anomaly scores remain stable even around timestamp 8000.

DCRNN and STGCN have several advantages and disadvantages during prediction and anomaly detection tasks. Therefore, we combine DCRNN and STGCN to detect anomalies.

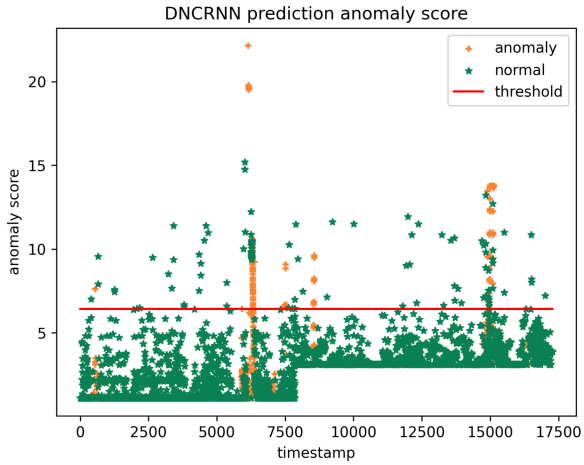

Fig. 3. DCRNN predictor's anomaly score distribution. The DCRNN method's anomaly scores range from 1 to 11 and exhibits a large span, indicating that the scores are widely spread and not concentrated.

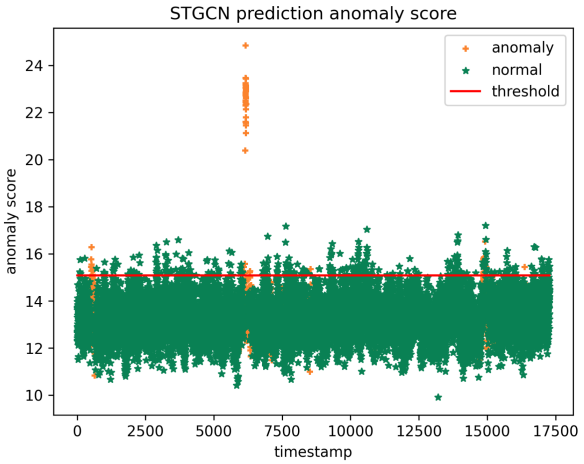

Fig. 4. STGCN predictors' anomaly score distribution. The STGCN anomaly scores range from 11 to 16. While the distribution is concentrated, the STGCN scores are significantly higher than those of DCRNN.

IV. Proposed Methodology

This section introduces the proposed method, elaborating on each module.

A. Overview

To obtain stable anomaly scores, we propose the MSTGAD method. MSTGAD uses the DTW distance to measure the node similarities and generate the prior graph. It uses the fully parameterized graph structure learner to generate a learnable graph and combines DCRNN and STGCN for ensemble prediction to achieve more accurate predictions and stable anomaly detection. As illustrated in Fig. 5, the MSTGAD framework comprises four key modules.

1) *DTW Prior Graph Learner:* To capture the delay correlation in time series data, we use DTW distance to measure time series similarities and obtain a similarity adjacency matrix. The DTW distance is invariant to time shifts compared with the regular Euclidean and cosine distances. Therefore, it can capture the delay correlations between the indicators.

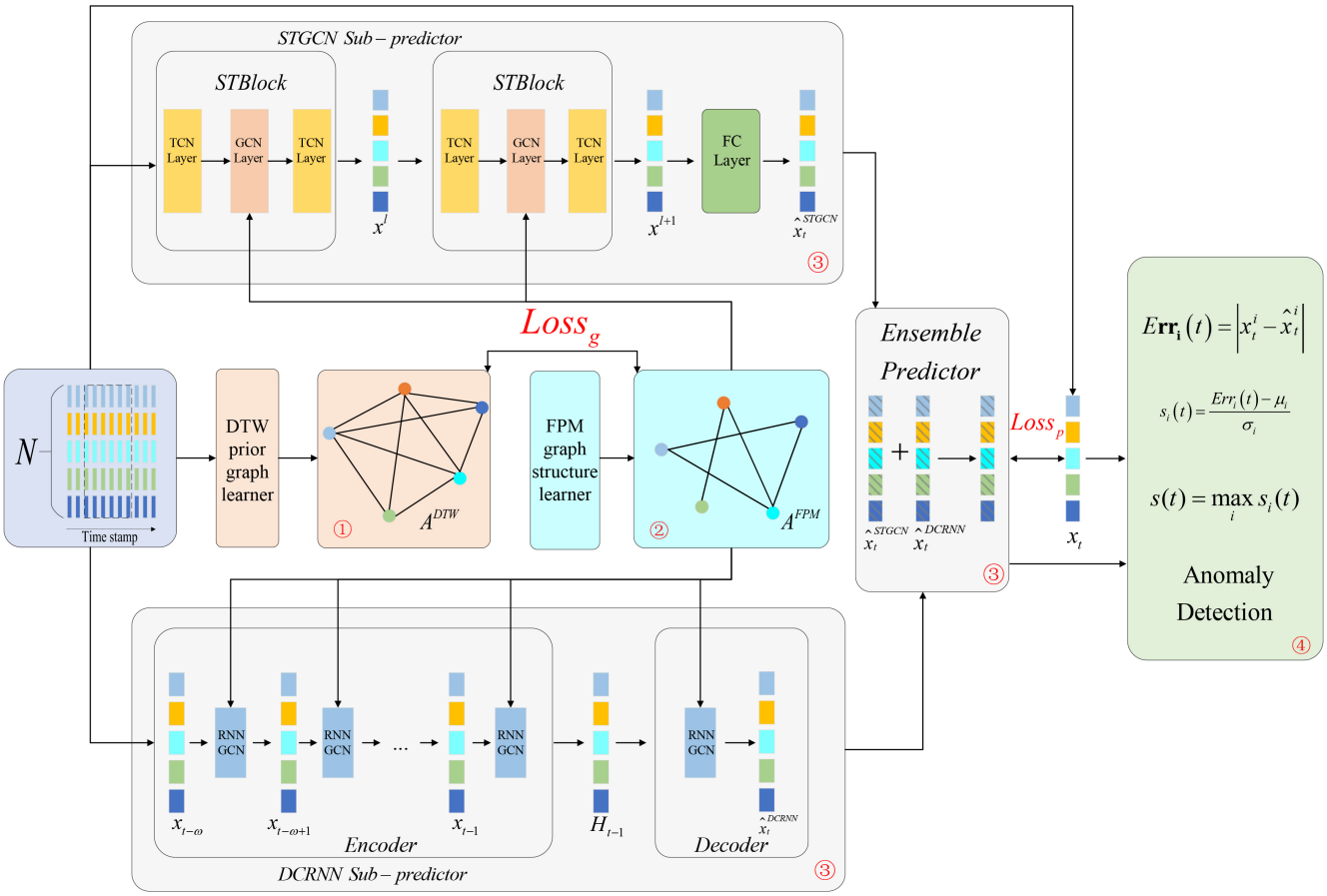

Fig. 5. MSTGAD framework. MSTGAD comprises four modules: the DTW prior graph learner, fully parameterized graph structure learner, STGCN and DRCNN-based ensemble predictor, and the anomaly score and threshold selection.

2) *Fully Parameterized Graph Structure Learner:* The graph structure learner exploits the full graph parameterization method to learn the relationships between indicators. In the full graph parameterization method, each element of the adjacency matrix A is directly modeled by a learnable and independent parameter, which is updated through backpropagation. This method generates a customized and optimal graph for the downstream task.
3) *STGCN- and DRCNN-Based Ensemble Predictor:* To avoid a single predictor pursuing high overall accuracy at the expense of discriminating anomaly scores, we adopt a combined approach that uses multiple spatiotemporal predictors. We divide the MTS into multiple subsequences using a sliding window. According to the learned adjacency matrix in the graph structure learner, the combined STGCN and DCRNN predictor uses the subsequence as the input for predicting the next timestamp's value.
4) *Anomaly Score and Threshold Selection:* The anomaly scores are derived from the prediction error between the prediction and the ground truth. A high anomaly score indicates that the subsequence may not follow normal patterns, increasing its likelihood of being an anomaly. When the anomaly score exceeds the selected threshold, we treat it as an anomaly. When the score is below the threshold, we treat it as normal.

The symbols used in this section are displayed in Nomenclature.

B. DTW Prior Graph Learner

Due to delay correlation in time series, the Euclidean distance may introduce noise to the prior graph [35]. Thus, the DTW distance [36] is more robust. Compared with the Euclidean and cosine distances, DTW is invariant to time shifts and can consider the time drift between two sequences. It also searches for the optimal alignment match using dynamic programming techniques.

Given two time series $Y = (y_1, y_2, \ldots, y_n)$ and $Z = (z_1, z_2, \ldots, z_m)$, the sequence distance matrix is $D \in R^{n\times m}$, where $D_{i,j} = |y_i - z_j|$. Thus, the cost matrix D_c is defined as follows:

$$D_c(i, j) = D_{i,j} + \min(D_c(i-1, j), D_c(i, j-1), D_c(i-1, j-1)). \quad (4)$$

Therefore, the final DTW distance is determined as follows:

$$\mathrm{DTW}(Y, Z) = D_c(n, m). \quad (5)$$

The optimal alignment match is the minimum function traversing path of $D_c(n, m)$. In this instance, the DTW distance

represents the best alignment gap between Y and Z, indicating the similarity between the time series.

Thus, we can obtain a similarity matrix $S^{\mathrm{DTW}} \in R^{N\times N}$ of all the indicators using the DTW distance

$$S_{i,j}^{\mathrm{DTW}} = \mathrm{DTW}(x^i, x^j) \tag{6}$$

where the values of the ith row and jth column in the similarity matrix is the DTW distance between the ith indicator $x^i(x_1^i, x_2^i, \ldots, x_M^i)$ and the jth indicator $x^j(x_1^j, x_2^j, \ldots, x_M^j)$. Then, we construct the adjacency matrix A^{DTW} of the prior graph using the DTW distance as follows:

$$A_{i,j}^{\mathrm{DTW}} = \begin{cases} 1, & S_{i,j}^{\mathrm{DTW}} < \varepsilon \\ 0, & \text{otherwise} \end{cases} \tag{7}$$

where ε is the DTW distance threshold that determines the sparsity of the adjacency matrix A^{DTW}.

C. Fully Parameterized Graph Structure Learner

The learnable graph structure's adjacency matrix consists of 0 and 1, which is nondifferentiable. As a result, the backpropagation algorithm struggles to efficiently compute the parameter gradient, thereby failing to jointly optimize the adjacency matrix and downstream task parameters. To address this issue, we use a reparameterization technique (i.e., Gumbel-Softmax), which represents a continuous distribution that can approximate samples from the class distribution. It enables the expression of discrete parameters using the continuous ones, simplifying the parameter gradient calculation. The Gumbel-Softmax process is defined as follows:

$$g = -\log(-\log(u)), \quad u \backsim \mathrm{Uniform}(0, 1) \tag{8}$$

$$z_1^{i,j} = \frac{\exp\left(\left(\log \pi_1^{i,j} + g^{i,j}\right)/\tau\right)}{\sum_{v\in\{0,1\}} \exp\left(\left(\log \pi_v^{i,j} + g^{i,j}\right)/\tau\right)} \tag{9}$$

where u represents the sample extracted from a Uniform(0, 1) distribution, $g^{i,j}$ follows a Gumbel distribution, and $\pi_1^{i,j}$ denotes the values of the ith row and jth column of probability matrix $\pi_1 \in R^{K\times K}$, indicating that the probability that node i is connected to j in the graph. In this instance, τ serves as a temperature parameter. As τ approaches 0, $z_1^{i,j}$ tends toward 0 or 1, and the Gumbel-Softmax distribution converges to the class distribution. Ultimately, the ith row and jth column in the adjacency matrix $A_{i,j}^{\mathrm{FPM}}$ are assigned the value $z_1^{i,j}$, causing $A_{i,j}^{\mathrm{FPM}}$ to be set to 1 with the probability of $\pi_1^{i,j}$.

In graph structure learners, we initialize the probability matrix π_1 randomly. Then, the adjacency matrix is derived from the probability matrix using Gumbel-Softmax sampling. Typically, the ensemble predictor uses the adjacency matrix for the prediction task. Thus, the probability matrix is updated using backpropagation to accomplish GSL.

D. Ensemble Predictor Based on STGCN and DCRNN

We design an ensemble predictor combining DCRNN and STGCN to overcome the issue that predictors cannot achieve high accuracy and a concentrated anomaly score distribution.

1) STGCN Subpredictor: STGCN [37] adopts a full convolution network, resolving the inherent issues of recursive networks such as RNN that require iterative training and accumulate errors during each step.

STGCN comprises several spatiotemporal convolution blocks (STBlock). These blocks comprise two temporal convolution networks (TCNs) and one GCN layer, forming a sandwich-like structure. The input X^l of the lth STBlock obtains output X^{l+1}, where X^1 is X_t. Therefore, the STBlock is defined as follows:

$$Y^{l+1} = \Gamma_1^l *_{\mathcal{T}} \mathrm{ReLU}\left(\Theta^l *_{\mathcal{G}} \left(\Gamma_0^l *_{\mathcal{T}} Y^l\right)\right) \tag{10}$$

$$\Gamma *_{\mathcal{T}} Y = P \odot \sigma(Q) \tag{11}$$

$$\Theta *_{\mathcal{G}} Y = \Theta(L)Y = \Theta(U\Lambda U^T)Y = U\Theta(\Lambda)U^T Y \tag{12}$$

$$L = I_n - D^{-\frac{1}{2}} A D^{-\frac{1}{2}} = U\Lambda U^T \tag{13}$$

where $\Gamma *_{\mathcal{T}} X$ is TCN and $\Theta *_{\mathcal{G}} X$ is GCN. Γ_0^l and Γ_1^l are the left and right temporal TCN kernels in the lth block. In addition, Θ^l denotes the spectral GCN kernel, and ReLU$(\cdot)$ is the activation function.

In (11), P and Q represent the output elements after the convolutional kernel, with $\odot$ representing the Hadamard product of elements. Furthermore, the sigmoid gate $\sigma(Q)$ controls which inputs P of the current state are related.

In (12), graph Fourier basis U denotes the eigenvector matrix of the normalized graph Laplacian L, and D is the degree matrix. In addition, Λ and $\Theta(\Lambda)$ are the diagonal matrix of the eigenvalues of L.

After stacking two STBlocks, we append a fully connected layer at the end as the output to map the last STBlock output to a predicted value $\widehat{x}_t^{\mathrm{STGCN}}$.

2) DCRNN Subpredictor: DCRNN [38] can capture temporal and spatial features. It uses diffusion convolution to capture spatial dependence and the encoder–decoder architecture to obtain temporal dependence. Therefore, we use DCRNN instead of GCN. Thus, DCRNN is defined as follows:

$$R_t = \mathrm{sigmoid}\left(W_R \circ \left[x_t || H_{(t-1)}\right] + b_R\right) \tag{14}$$

$$C_t = \tanh\left(W_C \circ \left[x_t || (R_t \odot H_{(t-1)})\right] + b_C\right) \tag{15}$$

$$U_t = \mathrm{sigmoid}\left(W_U \circ \left[x_t || H_{(t-1)}\right] + b_U\right) \tag{16}$$

$$H_{(t)} = U_t \odot H_{(t-1)} + (1 - U_t) \odot C_t \tag{17}$$

$$W_Q \circ Y = \sum_{l=0}^{L_2} \left(w_{l,1}^Q \left(D_O^{-1}A\right)^l + w_{l,2}^Q \left(D_I^{-1}A^T\right)^l\right) Y \tag{18}$$

where D_O and D_I are the out- and in-degree matrices, $\|$ is the concatenation operation, $w_{l,1}^Q$, $w_{l,2}^Q$, and b_Q are model parameters, and the diffusion degree L_2 is a hyperparameter.

During each step, $x_{t'}$ and node feature $H_{(t'-1)}$ enter the diffusion convolution layer to obtain the node feature $H_{(t')}$ at the next timestamp t'.

Then, the encoder updates the node feature $H_{(\cdot)}$ from the timestamp $t-\omega$ to $t-1$ and obtains the total node feature $H_{(t-1)}$ of the subsequence. In the decoder, the total node feature $H_{(t-1)}$ is used as the input. Subsequently, the predicted value $\widehat{x}_t^{\mathrm{DCRNN}}$ at the timestamp t is obtained, as shown in Fig. 5.

3) Ensemble Predictor: After obtaining the two predictions using the STGCN and DCRNN predictors, the results are combined. To balance the prediction result accuracy and anomaly score stability, we adjust their weights to enhance anomaly detection performance. The combined prediction result is defined as follows:

$$\widehat{x}_t = \lambda_1 \widehat{x}_t^{\text{STGCN}} + (1-\lambda_1)\widehat{x}_t^{\text{DCRNN}} \tag{19}$$

where λ_1 adjusts the two prediction results' weights.

E. Loss Function

Generally, the average absolute error is used as the loss function loss_p for the prediction task and can be defined as

$$\text{loss}_p = \frac{1}{K}\sum_{i=1}^{K} \left|\widehat{x}_t^i - x_t^i\right| \tag{20}$$

where $\widehat{x}_t^i$ and x_t^i represent the prediction and ground truth of the ith indicator at the timestamp t, respectively.

Furthermore, to improve the learned graph's quality and guarantee its robustness and generalization ability, we use the DTW prior graph as prior knowledge. Then, we introduce graph learning loss during model training to impose constraints on graph learning. Therefore, the graph learning loss function loss_g is expressed as the cross-entropy between prior knowledge A^{DTW} and the learned graph structure A^{FPM}

$$\text{loss}_g = \sum_{ij} -A_{i,j}^{\text{DTW}} \log A_{i,j}^{\text{FPM}} - \left(1-A_{i,j}^{\text{DTW}}\right)\log\left(1-A_{i,j}^{\text{FPM}}\right). \tag{21}$$

Thus, the total loss function is defined as follows:

$$\text{loss} = \text{loss}_p + \lambda_2 \text{loss}_g \tag{22}$$

where λ_2 is the regularization amplitude graph learning parameter.

F. Anomaly Scores and Threshold Selection

The purpose of anomaly detection is to identify abnormal situations that deviate from the norm according to the ground truth and prediction values. First, we calculate individual anomaly scores for each sensor. Then, these scores are combined to obtain the anomaly scores for each timestamp. Once the anomaly score at timestamp t exceeds the predetermined threshold, an anomaly has occurred at timestamp t.

The comparison between the ground truth and prediction values is conducted to obtain the prediction error $\text{Err}_{\text{i}}(t)$ of the sensor i at timestamp t

$$\text{Err}_{\text{i}}(t) = \left|x_t^i - \widehat{x}_t^i\right|. \tag{23}$$

To ensure metric scale consistency among sensors with varying value ranges, a standard normalization process is conducted on the prediction error as follows:

$$s_i(t) = \frac{\text{Err}_i(t) - \mu_i}{\sigma_i} \tag{24}$$

where μ_i and σ_i are the mean and standard deviation of $\text{Err}_i(t)$, respectively.

TABLE I
DATASET DESCRIPTION

Datasets	SWaT	WADI	MSL	SMAP
#Indicators	51	127	55	25
#Training	49500	76297	58317	135183
#Testing	45000	17280	73729	427617
#Anomaly rate	11.97%	5.99%	10.72%	13.13%

Then, the anomaly score at timestamp t is determined using the highest value among all the sensors

$$s(t) = \max_i s_i(t). \tag{25}$$

Grid search technology is pivotal for determining the optimal threshold. During grid search, the upper and lower bounds of the threshold are defined as the maximum and minimum values of $s(t)$, respectively. All possible thresholds are exhaustively searched with a step size of 0.01, and the threshold with the highest $F1$ score is selected as the best one. In addition, we use the point adjustment strategy to obtain the anomaly scores.

V. EXPERIMENT AND PERFORMANCE ANALYSIS

A. Datasets

We use four public real-world datasets; their statistics are displayed in Table I.

First, the SWaT dataset is derived from the water treatment test bench supervised by the Singapore Public Utilities Bureau. The data collection process lasted for 11 days and ran continuously 24 h a day, during which network traffic and values from all 51 sensors and actuators were recorded.

Second, the WADI dataset is obtained from a comprehensive WADI system composed of numerous pipelines. As an extension of the SWaT test platform, WADI shows the water treatment, storage, and distribution network more comprehensively and realistically. The dataset is collected over 16 consecutive days, with 14 dedicated to routine operations and the remaining two simulating attack scenarios. The whole test bed is equipped with 127 sensors and actuators.

Third, the Mars Science Laboratory (MSL) rover is a dataset comprising sensors and actuators from the NASA Mars rover. This dataset contains 55 indicators from 27 unique entities.

Finally, the Soil Moisture Active Passive (SMAP) satellite dataset comprises soil samples and telemetry data collected by NASA using the Mars probe. This dataset contains 25 indicators from 55 entities.

To handle substantial raw data, the SWaT and WADI datasets undergo a downsampling process every 10 s, capturing the median value. During this period, an anomaly is tagged once it occurs.

B. Experimental Setup

1) Metrics: We use precision (Prec), recall (Rec), and $F1$ score ($F1$) to evaluate the performance of our method and the

baselines

$$\text{Prec} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{26}$$

$$\text{Rec} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{27}$$

$$F1 = 2 * \frac{\text{Prec} * \text{Rec}}{\text{Prec} + \text{Rec}} \tag{28}$$

where TP, TN, FP, and FN are the number of true positives, true negatives, false positives, and false negatives, respectively.

2) Baselines: We compare our method with 12 machine and deep learning methods, including AE, IF, DAGMM, LSTM-NDT, LSTM-VAE, MAD-GAN, OmniAnomaly, USAD, MTAD-GAT, GDN, FuSAGNet, GTA, MEGLAD, MGCLAD, and GSLAD.

1) *AE:* In this method, the input data are reconstructed by an autoencoder, and the resulting reconstruction error is used as the anomaly score.
2) *IF:* Isolation forest is a tree-based anomaly detection algorithm that effectively identifies anomalies or outliers in datasets that differ significantly from the majority.
3) *DAGMM [28]*: This method combines deep autoencoders with Gaussian mixture models to capture the underlying distribution of normal data and identify anomalies based on reconstruction errors.
4) *LSTM-NDT [22]:* This method uses LSTM to capture contextual information and dependencies in sequential data for anomaly detection.
5) *LSTM-VAE [26]:* This method projects multimodal observations and time dependencies into a latent space and reconstructs the expected distribution using an LSTM-based VAE.
6) *MAD-GAN [24]:* This method uses LSTM as the underlying model in a GAN framework to capture the time series data temporal correlation.
7) *OmniAnomaly [27]:* This method is a priori-driven stochastic model used to detect timestamp anomalies, directly returning the reconstruction probability.
8) *USAD:* This method trains an encoder–decoder framework in an adversarial manner to achieve fast and efficient training.
9) *MTAD-GAT [30]:* This method regards the relationships between indicators as a complete graph and uses a GAT to detect anomalies.
10) *GDN [8]:* This method constructs a graph structure using pairwise cosine similarities between nodes and uses attention GNNs to learn dependencies between time series and execute prediction.
11) *FuSAGNet [10]:* This method learns the graph structure using the pairwise cosine similarity between recursive sensor embeddings. It obtains the input data's sparse representation using a sparse autoencoder and inserts it into a GAT to execute sensor behavior prediction.
12) *GTA [16]:* This method automatically learns the graph structure using a direct method and uses graph convolution and a transformer-based architecture to capture temporal dependencies.
13) *MEGLAD [19]:* This method uses three different GSL methods to learn the relationships between indicators from various perspectives. Then, it conducts time series prediction and anomaly detection based on the predicted values.
14) *MGCLAD [18]:* This method achieves anomaly detection by concurrently learning intra- and intersignal graph structures, capturing temporal context and dependencies between signals. It performs anomaly detection by comparing inputs and reconstructing outputs.
15) *GSLAD [20]:* This method uses a fully parameterized GSL method and uses diffusion graph convolution networks to detect anomalies.

3) Experimental Setting: Our main experimental parameter settings are shown in Table II. The DTW distance threshold ε is set to 0.5 for the SWaT dataset and 0.6 for the remaining three datasets according to STGODE [39]. In addition, the Gumbel-Softmax function is implemented by torch.nn.functional, where the temperature parameter τ is initialized to 1 and progressively approaches 0 during training. All the experiments were carried out in Python 3.8, PyTorch 1.10, and CUDA version 11.3 on a server equipped with an Intel[1] Xeon[1] Platinum 8255c CPU, NVIDIA RTX 3090 24-GB GPU, 90-GB memory, and 80 GB of hard disk space.

4) Baseline Method Parameter Settings: For the baseline methods, we set the sliding window size ω in GDN and FuSAGNet to 5, and GTA to 60.

C. Detection Performance

As shown in Table III, our method outperforms the baseline approaches significantly. Compared with the traditional CNN and LSTM anomaly detection methods (i.e., IF, DAGMM, LSTM-NDT, LSTM-VAE, MAD-GAN, OmniAnomaly, USAD), our method uses spatial and temporal GNNs to effectively extract the relationship between indicators. Compared with the latest GSL-based anomaly detection methods (such as GDN, FuSAGNet, GTA, MEGLAD, MGCLAD, and DPGLAD), our method achieves the highest average $F1$ score. This is because our method attains a better balance between prediction accuracy and anomaly scores, while enhancing the learned graph's quality using the DTW prior graph.

All the methods achieved low performances on the WADI dataset, because it has a longer length, more indicators, and lower anomaly rate than other datasets, as shown in Table II.

While MGCLAD demonstrates a superior performance in WADI, its reliance on GNN as an encoder for learning latent representations restricts the model's representational and generalization capacities. In contrast, our approach outperforms MGCLAD across the SWaT, SMAP, and MSL datasets, underscoring its broader applicability.

Moreover, to underscore our method's robustness, we perform multiple experiments and present a box plot, illustrating the $F1$ scores on all four datasets. The experimental results are depicted in Fig. 6. Notably, the $F1$ scores generated by our method consistently demonstrate a high level of coherence across all four datasets, underscoring its impressive stability.

[1]Registered Trademark.

TABLE II
EXPERIMENTAL PARAMETER SETTING

sliding window size ω	prediction weight parameter λ_1	graph loss regularization parameter λ_2	learning rate	batch size	epochs
12	0.5	1	0.005	64	30

TABLE III
PRECISION, RECALL, AND $F1$ SCORE ON THE FOUR DATASETS

Method	SWaT			WADI			SMAP			MSL			F1-average
	Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1	
AE	72.63	52.63	61.03	34.35	34.35	34.35	72.16	97.95	77.76	85.35	97.48	87.92	65.26
IF	96.2	73.15	83.11	62.41	61.55	61.98	44.23	51.05	46.71	56.81	67.4	59.84	62.91
DAGMM	27.46	69.52	39.37	54.44	26.99	36.09	63.34	99.84	71.24	75.62	98.03	81.12	56.95
LSTM-NDT	77.78	51.09	61.67	1.38	78.23	2.71	85.23	73.26	78.79	62.88	100	77.21	55.09
LSTM-VAE	96.24	59.91	73.85	87.79	14.45	24.82	71.64	98.75	75.55	85.99	97.56	85.37	64.89
MAD-GAN	98.97	63.74	77.54	41.44	33.92	37.3	81.57	92.16	86.54	85.16	99.3	91.69	73.26
OmniAnomaly	72.23	98.32	83.28	26.52	97.99	41.74	75.85	97.56	85.35	91.4	88.91	90.14	75.12
USAD	100	56	71.79	43.09	22.51	29.57	74.8	96.27	84.19	79.49	99.12	88.22	68.44
MTAD-GAT	21.03	64.46	31.71	11.72	30.55	16.94	79.91	99.91	88.8	79.17	98.24	87.68	56.28
GDN	99.35	68.12	80.82	97.5	40.19	56.92	74.8	98.91	85.18	93.08	98.92	95.91	79.71
FuSAGNet	98.7	72.6	83.7	82.9	47.8	60.7	9.3	67.9	16.5	80.6	98.9	88.9	62.45
GTA	93.9	85.7	89.6	79.6	79.4	79.5	89.11	91.76	90.41	91.04	91.17	91.11	87.65
MEGLAD	96.14	92.51	94.29	87.34	77.92	82.36	91.55	100	95.59	99.44	100	99.72	92.99
MGCLAD	94.81	89.11	91.87	98	87.84	**92.64**	93.42	92.71	93.07	95.1	95.89	95.49	93.26
GSLAD	96.77	93.53	95.13	78.2	90.2	83.8	94.56	100	97.2	97.5	100	98.7	93.71
MSTGAD	98.36	93.31	**95.77**	86.2	84.5	85.3	94.6	100	**97.2**	99.8	100	**99.9**	**94.54**

TABLE IV
PRECISION, RECALL, AND $F1$ SCORE OF ABLATION EXPERIMENTS

Method	SWaT			WADI			SMAP			MSL		
	Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1
COS+Ensemble	89.67	92.66	91.14	80.69	90.2	85.18	91.77	100	95.71	99.93	100	99.96
DTW+DCRNN	99.14	88.36	93.44	84.36	83.86	84.11	93.85	100	96.83	99.93	100	99.96
DTW+STGCN	99	68.3	80.84	58.2	72	64.4	93.03	100	96.39	99.08	93.73	96.33
w/o GSL loss	94.98	90.61	92.74	84.2	78.12	81.05	90.99	100	95.28	99.07	100	99.53
MSTGAD	98.36	93.31	**95.77**	86.2	84.5	**85.3**	94.6	100	**97.2**	99.8	100	**99.9**

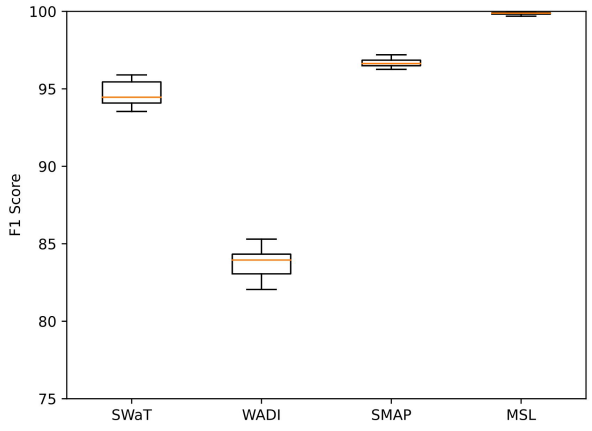

Fig. 6. $F1$ score distributions on the four datasets.

D. Ablation Experiments

To validate the DTW prior graph and ensemble predictor's efficacy, we conduct a series of ablation experiments.

1) Prior Graph Ablation Experiment: The DTW prior graph is replaced by a COS one, which compares sensor similarity using cosine distance. Then, we select the top-k most similar sensors to construct the prior graph

$$\cos\left(x^i, x^j\right) = \frac{x^i \bullet x^j}{\left\|x^i\right\| \bullet \left\|x^j\right\|} \tag{29}$$

$$A_{i,j}^{\mathrm{COS}} = 1,\ j \in \mathrm{topk}\left(\cos\left(x^i, x^j\right)\right). \tag{30}$$

The predictor is same as ensemble predictor (i.e., the COS + Ensemble function).

The experimental results are shown in Table IV. When the DTW prior graph is replaced by the COS one, performance decreases across all the datasets. This decrease in performance is because the DTW prior graph captures delay correlation, thereby avoiding the noise that may be present in the prior graph generated by the cosine distance. Therefore, our proposed anomaly detection method outperforms the COS prior graph. This further demonstrates our proposed DTW prior graph's effectiveness.

2) Predictor Ablation Experiment: To demonstrate the ensemble predictor's effectiveness, we conduct experiments using a single DCRNN or STGCN predictor with the DTW prior graph. As shown in Table IV, the single predictor's anomaly detection performance is lower than that of the ensemble predictor. This is because a single predictor cannot meet the requirements for high accuracy and stable anomaly scores during anomaly detection tasks simultaneously.

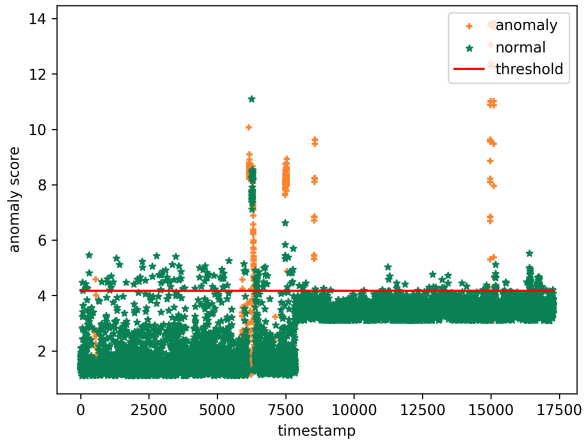

Fig. 7. Ensemble predictor anomaly score distribution. The ensemble predictor's anomaly scores range from 1 to 6. In comparison to the single DCRNN predictor, this predictor demonstrates more concentrated anomaly scores. Furthermore, it exhibits lower anomaly scores when compared with the single STGCN predictor.

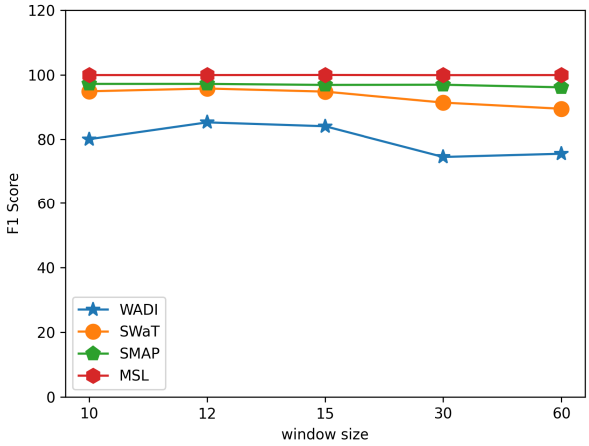

Fig. 8. Impact of window size.

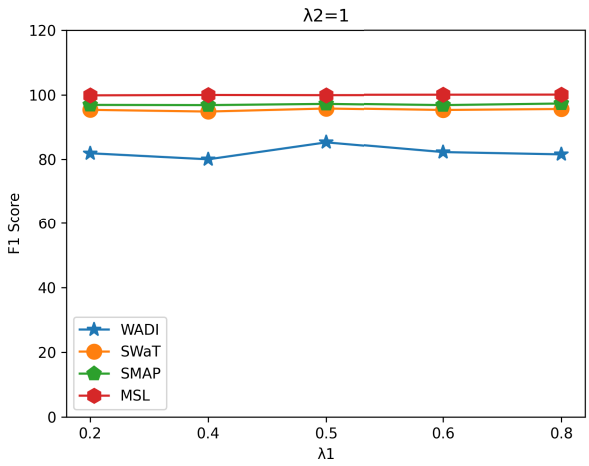

Fig. 9. Impact of prediction weight parameter.

Moreover, we highlight the overall anomaly score distribution of the ensemble predictor in the WADI dataset, as shown in Fig. 7. When comparing Figs. 7 and 3, the ensemble predictor concentrates the anomaly scores ranging from 1 to 6, whereas the single DCRNN one has anomaly scores ranging from 1 to 11. Similarly, in the comparison between Figs. 7 and 4, the ensemble predictor achieves lower prediction errors than the single STGCN one, which ranges from 11 to 16. The ensemble predictor can effectively balance the requirements of high prediction accuracy and stable anomaly scores, leading to improved overall performance.

3) Graph Learning Loss Function Ablation Experiment: To verify the graph learning loss function's effectiveness, we perform an ablation experiment where we exclude it from the total loss calculation, referred to as "w/o GSL loss."

The experimental results are shown in Table IV. Relying solely on prediction loss backpropagation to update the graph structure, without the support of prior graphs, results in a notable decrease in performance. This underscores the critical importance of incorporating graph learning loss into the overall function.

E. Parameter Influence

To assess our method's stability across various parameters, the impact of hyperparameters on model performance is analyzed.

1) Window Size: In this experiment, we set the window lengths w to 10, 12, 15, 30, and 60. The MSTGAD performance with different window sizes is displayed in Fig. 8.

The experimental results demonstrate that the window size influences the anomaly detection performance. For the SMAP and MSL datasets, the window size slightly impacts anomaly detection, resulting in a relatively stable $F1$ score. However, for the SWaT and WADI datasets, increasing the window size initially boosts the $F1$ score before declining. With a window size of 10, the historical information is insufficient to effectively support anomaly detection. Conversely, a window size of 60 can obscure anomalies within a large dataset, making anomaly detection more challenging.

2) Prediction Weight Parameter: We introduce prediction weight parameter on the four datasets. During this experiment, we adjust λ_1 from 0.2 to 0.8. The experimental results are shown in Fig. 9.

For the SWaT, SMAP, and MSL datasets, the prediction weight parameter λ_1 slightly impacts the performance. For the WADI dataset, MSTGAD with $\lambda_1 = 0.5$ exhibits the most effective performance. A value of $\lambda_1 = 0.5$ indicates that both the subpredictors contribute equally to the final prediction result. Moreover, the DCRNN subpredictor controls the overall prediction accuracy, while the STGCN one ensures the stability of the overall prediction results. Optimal anomaly detection performance is typically attained when these two aspects are in equilibrium.

3) DTW Distance Threshold in the Prior Graph: To assess the influence of the DTW distance's threshold on performance, we conduct an experiment to identify the optimal parameter. The DTW distance's threshold ranges from 0.2 to 1.0. A low threshold signifies a denser prior graph structure, whereas a high value indicates a sparse prior graph structure. We compare the impact of various threshold values by training the model on the four datasets and evaluating their performance.

TABLE V
PRECISION, RECALL, AND $F1$ SCORE FOR DIFFERENT DTW DISTANCE THRESHOLDS

Parameter ε	SWaT			WADI			SMAP			MSL		
	Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1	Prec	Rec	F1
0.2	99.16	88.8	93.69	85.38	78.11	81.59	91.77	100	95.71	99.07	100	99.53
0.4	98.29	93.13	95.64	83.03	83.86	83.44	93.5	100	96.64	99.93	100	99.96
0.5	98.36	93.31	**95.77**	85.94	78.11	81.84	93.15	100	96.45	99.69	100	99.84
0.6	98.82	90.61	94.53	86.2	84.5	**85.3**	94.6	100	**97.2**	99.8	100	**99.9**
0.7	99.18	90.41	94.59	75.97	84.85	80.16	92.57	100	96.14	99.26	100	99.62
0.8	98.97	90.01	94.28	80.73	83.86	82.27	92.45	100	96.08	99.51	100	99.75
1.0	92.86	91.21	92.02	82.79	78.11	80.38	90.55	100	95.04	99.19	100	99.59

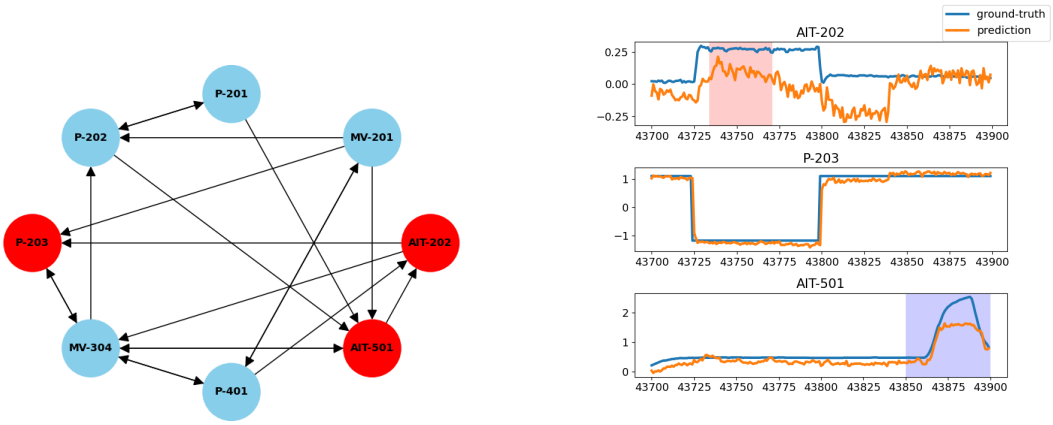

Fig. 10. Partial graph structure learned using the SWaT dataset (left). The ground truth and prediction values of the related sensors (right).

The experimental results are shown in Table V. When the DTW distance exceeds 1 or falls below 0.2, a notable deterioration in the model's performance occurs. This is primarily due to the fact that overly sparse or dense prior graphs do not effectively facilitate GSL. An excessively sparse prior adjacency matrix decreases the essential information required for guiding the graph structure learner to understand the relationships between the indicators. Conversely, an overly dense adjacency matrix contains redundant information, making it challenging for the graph structure learner to differentiate crucial edges or relationships among indicators. Therefore, the DTW distance threshold is set to 0.5 for the SWaT dataset and 0.6 for the remaining three datasets.

F. Case Study

We conduct a case study to evaluate the learned graph, as shown in Fig. 10. Fig. 10 (Left) is a partial graph structure learned using our method, and Fig. 10 (Right) is the ground truth and prediction of related sensors obtained with our method. Specifically, the sensor AIT-202 was attacked from the timestamp 43 734 to 43 771, which are marked in pink. Due to the interactions between sensors during the water treatment process, AIT-202 was attacked, leading to the shutdown of P-203 in the dosing pump. Due to several water treatment steps, AIT-501, an osmotic conductivity analyzer, was finally affected at timestamp 43 850, as shown in Fig. 10 (Right).

Our model accurately predicts the changes in AIT-501, thereby demonstrating our model's effectiveness. Moreover, Fig. 10 (Left) accurately shows the relationship between the three sensors in the learned graph.

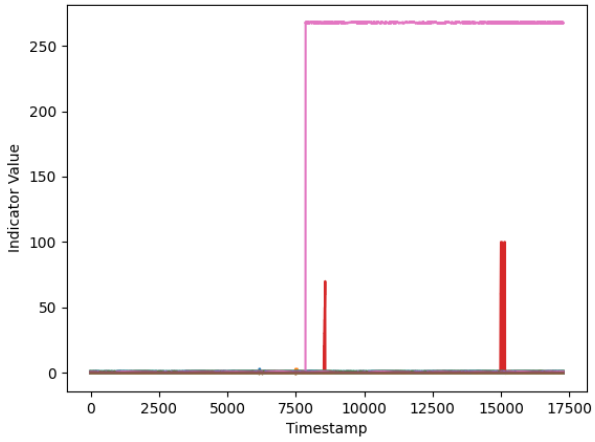

Fig. 11. Indicator values of the WADI dataset.

VI. CONCLUSION AND FUTURE WORK

In this article, we propose the MSTGAD method. We incorporate the DTW distance metric to measure the similarity between time series pairs, effectively capturing the delay correlations in MTS and obtaining a more accurate prior graph. Simultaneously, using an ensemble predictor with multiple graph convolutions, we effectively predict sensor behavior and significantly improve anomaly score stability. Compared with the baselines on the four public datasets, our proposed method achieves the best performance. In future work, we can consider more efficient GSL strategies to further enhance model performance.

APPENDIX

The WADI dataset includes 127 indicators. Fig. 11 illustrates the sensor values in the WADI dataset. The indicator (2B_AIT_002_PV) exhibits a sharp increase from 0.05 to 267 at timestamp 8000.

REFERENCES

[1] Y. Liu, Q. Li, and K. Wang, "Revealing the degradation patterns of lithium-ion batteries from impedance spectroscopy using variational auto-encoders," *Energy Storage Mater.*, vol. 69, May 2024, Art. no. 103394.

[2] X. Yu, T. Ai, and K. Wang, "Application of nanogenerators in acoustics based on artificial intelligence and machine learning," *APL Mater.*, vol. 12, no. 2, Feb. 2024, Art. no. 020602.

[3] S. He, Z. Li, J. Wang, and N. N. Xiong, "Intelligent detection for key performance indicators in industrial-based cyber-physical systems," *IEEE Trans. Ind. Informat.*, vol. 17, no. 8, pp. 5799–5809, Aug. 2021.

[4] S. He et al., "Fine-grained multivariate time series anomaly detection in IoT," *Comput., Mater. Continua*, vol. 75, no. 3, pp. 5027–5047, 2023.

[5] J. Zhou et al., "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, Jun. 2020.

[6] Y. Zhu et al., "A survey on graph structure learning: Progress and opportunities," 2021, *arXiv:2103.03036*.

[7] D. Chai, L. Wang, and Q. Yang, "Bike flow prediction with multi-graph convolutional networks," in *Proc. 26th ACM SIGSPATIAL Int. Conf. Adv. Geographic Inf. Syst.*, 2018, pp. 397–400, doi: 10.1145/3274895.3274896.

[8] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, May 2021, pp. 4027–4035.

[9] R. Gao, W. He, L. Yan, D. Liu, Y. Yu, and Z. Ye, "Hybrid graph transformer networks for multivariate time series anomaly detection," *J. Supercomput.*, vol. 80, no. 1, pp. 642–669, Jan. 2024.

[10] S. Han and S. S. Woo, "Learning sparse latent graph representations for anomaly detection in multivariate time series," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2022, pp. 2977–2986.

[11] W. Zhang, P. He, C. Qin, F. Yang, and Y. Liu, "A graph attention network-based model for anomaly detection in multivariate time series," *J. Supercomput.*, vol. 80, no. 6, pp. 8529–8549, Apr. 2024.

[12] J. Wang, S. Shao, Y. Bai, J. Deng, and Y. Lin, "Multiscale wavelet graph AutoEncoder for multivariate time-series anomaly detection," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–11, 2023.

[13] S. He, G. Li, J. Wang, K. Xie, and P. K. Sharma, "Uni-directional graph structure learning-based multivariate time series anomaly detection with dynamic prior knowledge," *Int. J. Mach. Learn. Cybern.*, vol. 2, pp. 1–17, May 2024.

[14] Z. Zhang, Z. Geng, and Y. Han, "Graph structure change-based anomaly detection in multivariate time series of industrial processes," *IEEE Trans. Ind. Informat.*, vol. 20, no. 4, pp. 6457–6466, Apr. 2024.

[15] H. Pang et al., "Asymptotic consistent graph structure learning for multivariate time-series anomaly detection," *IEEE Trans. Instrum. Meas.*, vol. 73, pp. 1–10, 2024.

[16] Z. Chen, D. Chen, X. Zhang, Z. Yuan, and X. Cheng, "Learning graph structures with transformer for multivariate time-series anomaly detection in IoT," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9179–9189, Jun. 2021.

[17] X. Zhou, C. Dai, W. Wang, and T. Qiu, "Global–local association discrepancy for multivariate time series anomaly detection in IIoT," *IEEE Internet Things J.*, vol. 11, no. 7, pp. 11287–11297, May 2024.

[18] S. Qin, L. Chen, Y. Luo, and G. Tao, "Multiview graph contrastive learning for multivariate time-series anomaly detection in IoT," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 22401–22414, Apr. 2023.

[19] S. He, G. Li, Q. Guo, and K. Xie, "Multi-graph structure learning-based multivariate time series anomaly detection with extended prior knowledge," in *Proc. 27th Int. Conf. Comput. Supported Cooperat. Work Design (CSCWD)*, May 2024, pp. 109–114.

[20] S. He et al., "Graph structure learning-based multivariate time series anomaly detection in Internet of Things for human-centric consumer applications," *IEEE Trans. Consum. Electron.*, vol. 2, no. 1, pp. 1–15, May 2024.

[21] H. Yu et al., "Regularized graph structure learning with semantic knowledge for multi-variates time-series forecasting," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 2362–2368.

[22] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Söderström, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, Jul. 2018, pp. 387–395.

[23] C. Zhang et al., "A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 1409–1416.

[24] D. Li, D. Chen, B. Jin, L. Shi, J. Goh, and S.-K. Ng, "MAD-GAN: Multivariate anomaly detection for time series data with generative adversarial networks," in *Proc. Int. Conf. Artif. Neural Netw. (ICANN)*, 2019, pp. 703–716.

[25] K.-J. Jeong, J.-D. Park, K. Hwang, S.-L. Kim, and W.-Y. Shin, "Two-stage deep anomaly detection with heterogeneous time series data," *IEEE Access*, vol. 10, pp. 13704–13714, 2022.

[26] D. Park, Y. Hoshi, and C. C. Kemp, "A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1544–1551, Jul. 2018.

[27] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2019, pp. 2828–2837.

[28] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–10.

[29] Q. He, Y. J. Zheng, C. L. Zhang, and H. Y. Wang, "MTAD-TF: Multivariate time series anomaly detection using the combination of temporal pattern and feature pattern," *Complexity*, vol. 2020, pp. 1–9, Oct. 2020.

[30] H. Zhao et al., "Multivariate time-series anomaly detection via graph attention network," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2020, pp. 841–850.

[31] D. Scheinert, A. Acker, L. Thamsen, M. K. Geldenhuys, and O. Kao, "Learning dependencies in distributed cloud applications to identify and localize anomalies," in *Proc. IEEE/ACM Int. Workshop Cloud Intell.*, May 2021, pp. 7–12.

[32] L. Zhou, Q. Zeng, and B. Li, "Hybrid anomaly detection via multihead dynamic graph attention networks for multivariate time series," *IEEE Access*, vol. 10, pp. 40967–40978, 2022.

[33] W. Chen, L. Tian, B. Chen, L. Dai, Z. Duan, and M. Zhou, "Deep variational graph convolutional recurrent network for multivariate time series anomaly detection," in *Proc. Int. Conf. Mach. Learn.*, vol. 162, 2022, pp. 3621–3633.

[34] W. Zhang, C. Zhang, and F. Tsung, "GRELEN: Multivariate time series anomaly detection from the perspective of graph relational learning," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2022, pp. 2390–2397.

[35] K. Choi, J. Yi, C. Park, and S. Yoon, "Deep learning for anomaly detection in time-series data: Review, analysis, and guidelines," *IEEE Access*, vol. 9, pp. 120043–120065, 2021.

[36] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. ASSP-26, no. 1, pp. 43–49, Feb. 1978.

[37] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, Jul. 2018, pp. 3634–3640.

[38] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *Proc. 6th Int. Conf. Learn. Represent.*, 2018, pp. 1–24.

[39] Z. Fang, Q. Long, G. Song, and K. Xie, "Spatial-temporal graph ODE networks for traffic flow forecasting," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Min.*, 2021, pp. 364–373.

Shiming He received the B.S. degree in information security and the Ph.D. degree in computer science and technology from Hunan University, Changsha, China, in 2006 and 2013, respectively.

She is currently an Associate Professor with the School of Computer and Communication Engineering, Changsha University of Science and Technology, Changsha. Her research interests include machine learning, data analysis, and anomaly detection.

Qingqing Guo received the B.S. degree from Hunan Agricultural University, Changsha, China, in 2022. He is currently pursuing the M.S. degree in computer technology with Changsha University of Science and Technology, Changsha.

His research interests include deep learning, data analysis, and anomaly detection.

Genxin Li received the B.S. degree from Jiangxi Science and Technology Normal University, Nanchang, China, in 2022. He is currently pursuing the M.S. degree in computer science and technology with Changsha University of Science and Technology, Changsha, China.

His research interests include deep learning, data analysis, and anomaly detection.

Kun Xie (Member, IEEE) received the Ph.D. degree in computer application from Hunan University, Changsha, China, in 2007.

She has published more than 100 articles in major journals and conference proceedings, including journals such as IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, and IEEE TRANSACTIONS ON SERVICES COMPUTING, and conferences, including SIGMOD, INFOCOM, ICDCS, SECON, DSN, and IWQoS. Her research interests include cover network measurement, network security, big data, and AI.

Pradip Kumar Sharma (Senior Member, IEEE) received the bachelor's degree in information technology in 2009, the Master of Technology degree in computer science in 2014, and the Ph.D. degree in secure computing in 2019.

He is a highly Esteemed Academician and Researcher known for his expertise in Cybersecurity and Artificial Intelligence. He is currently serving as an Associate Professor of Cybersecurity & AI at the Department of Computing Science, University of Aberdeen, U.K. His academic journey is characterised by a steadfast pursuit of excellence. His professional background encompasses roles such as Research Fellow and Software Engineer, reflecting a blend of academic and industry experience. Throughout his career, he has shown a strong interest in advancing research in areas such as privacy-aware AI, blockchain, edge computing, the IoT security, and critical infrastructure security, earning recognition through various research projects and grants.

Dr. Sharma received the awards from EPSRC and Innovate U.K.